

Sistemi Operativi

Marco Tartaglione

7 ottobre 2026

Indice

1 Domande di teoria	3
1.1 Confrontare l'organizzazione monolitica con quella basata su microkernel	3
1.2 Descrivere l'immagine di un processo nel SO Unix	3
1.3 Il processo in Unix: caratteristiche e rappresentazione	3
1.4 Descrivere i meccanismi del SO all'invocazione di una system call	4
1.5 La comunicazione tra processi in Unix	5
1.6 Descrivi la sincronizzazione tra processi in Unix	5
1.7 Descrivere l'accesso e la gestione dei file da parte di processi Unix	5
1.8 Descrivere le tecniche di allocazione della memoria centrale	6
1.9 Descrivere la paginazione	7
1.10 Descrivere la segmentazione	7
1.11 Descrivere la memoria virtuale	8
1.12 Descrivere le operazioni svolte dopo un Page Fault	9
1.13 Descrivere la sovrallocazione e gli algoritmi di sostituzione	9
1.14 Descrivere i metodi di accesso e di allocazione nel file system	10
1.15 Descrivere l'organizzazione fisica del file system in Unix	10
1.16 Descrivi lo scheduling della CPU	11
1.17 Descrivere i passaggi di sostituzione ed elaborazione nella Shell Bash	12
1.18 Descrivere i Monitor e variabili condizione	12
1.19 Descrivi le operazioni e semantiche delle variabili condizione	13
1.20 Esporre possibili soluzioni al problema della mutua esclusione	13
1.21 Definizione e uso di semaforo	14
1.22 Descrizione e confronto tra gestione a controllo di programma e gestione a interruzioni di dispositivi di I/O	14

1 Domande di teoria

1.1 Confrontare l'organizzazione monolitica con quella basata su microkernel

Esistono due tipi principali di strutture per un sistema operativo: **monolitica** e **microkernel**.

La prima presenta il sistema sotto forma di un unico eseguibile, nel quale le singole procedure rappresentano i vari componenti. La comunicazione tra questi avviene attraverso chiamate a *procedura*, il che favorisce un basso costo di interazione tra i componenti. Tuttavia, ciò comporta una maggiore difficoltà di manutenzione, portabilità, affidabilità ed estendibilità.

I sistemi a microkernel, invece, presentano una struttura del kernel estremamente semplice, ridotta a poche funzionalità di base, e arricchita da servizi gestiti tramite processi utente. Un sistema operativo di questo tipo risulta generalmente più lento (a causa dell'elevato utilizzo di *system call*) ma garantisce una migliore manutenzione e maggiore sicurezza.

1.2 Descrivere l'immagine di un processo nel SO Unix

Con immagine di un processo si intende l'insieme delle **aree di memoria** e delle **strutture dati** associate a un processo. A ciascuna di queste possono essere associate alcune proprietà:

- **Swappable**: la parte dell'immagine può essere spostata in memoria secondaria in caso di necessità.
- **Non swappable**: la parte dell'immagine non può essere spostata in memoria secondaria.
- **Kernel**: la parte dell'immagine può essere eseguita e modificata solo dal kernel.
- **User**: la parte dell'immagine può essere eseguita e modificata dal processo utente.

I principali componenti dell'immagine di un processo sono:

- **Process structure** [kernel, non swappable]: contiene le informazioni necessarie per la gestione del processo da parte del sistema operativo (ID, stato, priorità di scheduling, ecc.)
- **User structure** [kernel, swappable]: contiene informazioni per la gestione del processo quando è caricato in memoria (registri della CPU, tabella dei file aperti, ecc.)
- **Text structure** [kernel, non swappable]: puntatore alla *text structure* contenente il riferimento all'area di memoria associata al codice eseguibile del processo
- **Codice** [user, swappable]: puntato dalla *text structure*
- **Area dati globali** [user, swappable]: riferimento all'area di memoria contenente i dati globali del processo
- **Stack e Heap** [user, swappable]: riferimento alle aree di stack e heap del processo utente
- **Stack del kernel** [kernel, swappable]: riferimento all'area di stack utilizzata dal kernel durante l'esecuzione del processo

1.3 Il processo in Unix: caratteristiche e rappresentazione

In Unix, il **processo** rappresenta l'unità fondamentale di esecuzione (o computazione pesante). Ogni processo dispone di un proprio *spazio di indirizzamento locale*, che include codice (di tipo rientrante, ovvero condivisibile con altri processi) e dati (separati dal codice).

Ogni processo è descritto da una struttura chiamata **Process Control Block** (PCB), che contiene tutte le informazioni necessarie per la sua gestione da parte del sistema operativo. Il PCB è costituito principalmente da due componenti:

- **Process structure**: include informazioni descrittive del processo (ID, priorità, processo padre, puntatori alla *text structure* e alla *user structure*, stato, ecc.)
- **User structure**: include informazioni relative all'esecuzione del processo (stato dei registri della CPU, tabella dei file aperti, gestione dei segnali, ecc.)

Stati di un processo: la *Process structure* tiene traccia dello stato corrente del processo, che può assumere i seguenti valori:

- **Init:** il processo è stato inizializzato e caricato in memoria
- **Ready:** il processo è pronto per essere eseguito
- **Running:** il processo è attualmente in esecuzione sulla CPU
- **Sleeping:** il processo è in attesa di un evento (es. I/O)
- **Swapped:** il processo, o parte di esso, è stato spostato in memoria secondaria
- **Zombie:** il processo è terminato, ma non ancora raccolto dal processo padre
- **Terminated:** il processo è terminato e le sue risorse sono state liberate

Nel caso di uno stato **Swapped**, il processo è stato trasferito in memoria secondaria tramite lo **swapper**, ovvero lo *scheduler* a medio termine, per liberare spazio in RAM.

Classificazione delle aree di memoria:

Ogni area associata a un processo può avere le seguenti caratteristiche:

- **Swappable:** può essere trasferita in memoria secondaria
- **Non swappable:** non può essere trasferita in memoria secondaria
- **Kernel:** accessibile ed eseguibile solo in modalità kernel
- **User:** accessibile ed eseguibile in modalità utente

Componenti dell'immagine di un processo:

Oltre al PCB, l'immagine di un processo comprende tutte le aree di memoria che lo compongono:

- **Text structure** [kernel, non swappable]: puntatore alla struttura contenente informazioni sull'area di memoria del codice eseguibile
- **Codice** [user, swappable]: area contenente il codice macchina del processo, puntata dalla *text structure*
- **Area dati globali** [user, swappable]: contiene i dati globali e statici del processo
- **Stack e Heap** [user, swappable]: aree dinamiche utilizzate per l'allocazione temporanea (stack) e dinamica (heap)
- **Stack del kernel** [kernel, swappable]: area di stack usata dal kernel quando il processo esegue in modalità kernel (es. durante una system call)

Nota sulla modalità kernel:

Per **modalità kernel** si intende il contesto di esecuzione privilegiato del sistema operativo. Un processo utente vi entra tramite una *system call* (chiamata di sistema), che provoca un cambio di contesto e consente l'esecuzione di codice del kernel per conto del processo. Al termine dell'operazione, il controllo viene restituito al processo in **modalità utente**.

1.4 Descrivere i meccanismi del SO all'invocazione di una system call

All'invocazione di una *system call*, un processo utente richiede l'intervento del sistema operativo per eseguire un'operazione privilegiata. Questa richiesta avviene tramite un'istruzione apposita (es. **syscall**), che provoca un **passaggio di controllo** dal processo utente al kernel.

Durante questo passaggio:

- Viene salvato lo stato corrente del processo (registri, program counter, ecc.)
- Il processore imposta un **flag di protezione**, attivando la **kernel mode**
- L'esecuzione viene trasferita a una specifica routine del sistema operativo
- Il sistema operativo esegue le operazioni richieste, con privilegi elevati

Una volta completata la system call:

- Lo stato salvato del processo viene ripristinato
- Il flag di protezione viene disattivato, riportando la CPU in **user mode**
- L'esecuzione del processo riprende dal punto successivo alla chiamata

Tale meccanismo garantisce un'interfaccia sicura e controllata tra i processi utente e le risorse protette gestite dal sistema operativo.

1.5 La comunicazione tra processi in Unix

Nei sistemi Unix, i processi possono comunicare tra loro attraverso due modalità principali:

Segnali:

Un segnale è un evento software che notifica a un processo che è avvenuta una certa condizione. Un processo può inviare un segnale a un altro tramite la system call `kill(pid, SIG)`, dove `SIG` è il segnale, e può definire come reagire a un segnale tramite `signal(SIG, handler)`, dove `handler` può essere:

- un puntatore a una funzione (gestione personalizzata)
- `SIG_DFL` (gestione di default, obbligatoria per `SIGKILL` e `SIGSTOP`)
- `SIG_IGN` (ignorare il segnale)

Pipe/Fifo:

- **Pipe**: canali unidirezionali di comunicazione tra processi **imparentati** (es. padre-figlio). Si creano con la system call `pipe(int fd[2])`, che restituisce due file descriptor: uno per la lettura (`fd[0]`) e uno per la scrittura (`fd[1]`).
- **FIFO** (First In, First Out): simili alle pipe, ma **persistenti** e **accessibili tramite nome**. Si creano con `mkfifo(const char *pathname, mode_t mode)` e sono rappresentate come file speciali nel file system.

Entrambi i meccanismi operano secondo la logica FIFO e sono soggetti a blocchi e sincronizzazione se uno degli estremi (lettura o scrittura) non è pronto. Le **FIFO**, a differenza delle pipe, permettono comunicazioni tra processi indipendenti e supportano la gestione dei permessi (utente, gruppo, altri) come i normali file.

1.6 Descrivere la sincronizzazione tra processi in Unix

La sincronizzazione è un meccanismo che permette di coordinare l'esecuzione di più processi che possono interagire tra loro. In Unix, modello ad ambiente locale, i processi possono sincronizzarsi esclusivamente attraverso l'utilizzo di eventi detti **segnali**.

I segnali sono notifiche asincrone inviate ai processi per indicare che si è verificato un evento. Ogni processo può inviare un segnale a un altro tramite la system call `kill(pid, segnale)` e può definire un comportamento personalizzato alla ricezione di un segnale utilizzando `signal(segnale, handler)`. Il parametro `handler` può essere:

- un puntatore a una funzione (signal handler),
- la costante `SIG_IGN` per ignorare il segnale,
- oppure `SIG_DFL` per mantenere il comportamento di default.

Alcuni segnali, come `SIGKILL` e `SIGSTOP`, non possono essere intercettati o ignorati: il loro comportamento è fisso e serve a garantire il controllo da parte del sistema operativo.

1.7 Descrivere l'accesso e la gestione dei file da parte di processi Unix

In Unix, i file sono gestiti come sequenze di byte non strutturate e l'accesso può avvenire in modo sequenziale. Quando un processo apre un file, il sistema operativo associa a quel file una struttura detta *I/O pointer* (puntatore di file), che registra la posizione corrente all'interno del file. Questo puntatore è indipendente per ciascun processo che apre il file.

Ogni processo mantiene una propria **tabella dei file aperti** (all'interno della *User Structure*), in cui a ciascun file aperto è assegnato un **file descriptor** (FD), che è un indice intero nella tabella stessa.

Il sistema operativo mantiene due strutture fondamentali a livello di kernel:

- **Tabella dei file di sistema**: per ogni file aperto contiene informazioni come l'i-node associato, le modalità di accesso (lettura/scrittura), e il puntatore al file (I/O pointer). Se un file viene aperto più volte è presente in più entry con attributi differenti.

- **Tabella dei file attivi:** tiene traccia degli i-node caricati in memoria. L'i-node contiene gli attributi del file (dimensione, permessi, puntatori ai blocchi su disco, ecc.). In questo modo si evitano accessi ripetuti al disco.

Unix fornisce diverse system call per la gestione dei file:

- **open():** apre un file e restituisce un file descriptor
- **close():** chiude un file precedentemente aperto
- **creat():** crea un nuovo file
- **read():** legge dati da un file
- **write():** scrive dati su un file
- **lseek():** sposta il puntatore di lettura/scrittura nel file
- **link():** crea un collegamento a un file esistente, incrementando il contatore di link dell'i-node
- **unlink():** rimuove un collegamento a un file; se il numero di link scende a zero e non ci sono file aperti, il file viene eliminato
- **access():** verifica i permessi di accesso a un file
- **stat():** restituisce informazioni dettagliate su un file.

1.8 Descrivere le tecniche di allocazione della memoria centrale

Le tecniche di allocazione della memoria centrale possono essere classificate in due categorie principali:

- **Allocazione contigua**
- **Allocazione non contigua**

Allocazione contigua:

In questo modello, ogni processo viene caricato in un'unica area contigua di memoria fisica. Si suddivide in:

- **Ripartizione singola:** l'intera memoria utente è riservata a un solo processo alla volta, oltre al sistema operativo. È una tecnica obsoleta, utilizzata nei primi sistemi batch.
- **Ripartizione multipla:** la memoria è suddivisa in più partizioni, ognuna delle quali può ospitare un processo. Può essere:
 - **A partizioni fisse:** la memoria è divisa in partizioni di dimensione predefinita. I processi vengono assegnati alla partizione più adatta (*fit*).
 - **A partizioni variabili:** le partizioni vengono create dinamicamente in base alla dimensione dei processi. L'allocazione richiede tecniche di gestione dello spazio libero (first fit, best fit, worst fit) e può generare frammentazione esterna.

Allocazione non contigua:

In queste tecniche, i processi possono essere suddivisi in più blocchi caricati in posizioni non contigue della memoria fisica. Le principali soluzioni sono:

- **Paginazione:** la memoria fisica è divisa in blocchi di dimensione fissa detti *frame*, e la memoria logica del processo in *pagine* della stessa dimensione. Ogni pagina viene mappata su un frame fisico tramite una **tabella delle pagine**, mantenuta dal sistema operativo. Elimina la frammentazione esterna, ma può introdurre frammentazione interna.
- **Segmentazione:** lo spazio di indirizzamento del processo è suddiviso in segmenti logici (es. codice, dati, stack), ciascuno con dimensione variabile e significato semantico. Ogni segmento viene allocato separatamente in memoria fisica. Viene usata una **tabella dei segmenti** per gestire gli accessi.
- **Segmentazione paginata:** combina i vantaggi di paginazione e segmentazione. Ogni segmento viene suddiviso in pagine, che vengono poi mappate su frame fisici. Richiede l'uso combinato di tabella dei segmenti e tabelle delle pagine per ciascun segmento.

1.9 Descrivere la paginazione

La **paginazione** è una tecnica di gestione della memoria centrale che consente l'allocazione non contigua dello spazio logico di un processo. Essa divide:

- lo **spazio fisico** (RAM) in blocchi di dimensione fissa chiamati *frame*;
- lo **spazio logico** (indirizzamento del processo) in blocchi della stessa dimensione chiamati *pagine*.

Questa corrispondenza consente di mappare indirizzi logici contigui su frame fisici non contigui, riducendo la **frammentazione esterna** e limitando quella **interna**.

Un indirizzo logico ha la forma $\langle p, d \rangle$, dove:

- p è il numero di pagina logica;
- d è l'offset all'interno della pagina.

L'indirizzo fisico risultante sarà $\langle f, d \rangle$, dove f è il frame fisico associato alla pagina p .

La mappatura è gestita dalla **tabella delle pagine**, mantenuta per ogni processo. Ogni voce della tabella associa una pagina logica a un frame fisico. Se la tabella è in memoria centrale, l'accesso può essere ottimizzato tramite la **TLB** (*Translation Lookaside Buffer*), una cache hardware delle associazioni recenti. Ogni voce della tabella delle pagine può includere:

- un **bit di validità** per indicare se la voce è attiva (cioè se la pagina è in RAM);
- un **bit di protezione** per definire i permessi di accesso (es. sola lettura);
- un **dirty bit** per indicare se la pagina è stata modificata (utile per decidere se salvarla su disco durante uno *swap-out*).

Quando un processo accede a una pagina che non è presente in memoria fisica (bit di validità = 0), si verifica un **page fault**. In tal caso:

1. Il controllo passa al sistema operativo.
2. Il **pager** (componente del SO) sceglie una pagina da caricare dalla memoria secondaria (swap space).
3. Se la memoria è piena, una pagina esistente può essere rimossa (eventualmente salvandola su disco se è stata modificata).
4. Viene aggiornata la tabella delle pagine e il processo viene riattivato.

Per evitare tabelle troppo grandi (soprattutto nei sistemi a 64 bit), si possono usare tabelle delle pagine **a più livelli**. Una tabella di primo livello contiene puntatori a tabelle di secondo livello, che contengono le vere associazioni pagina-frame. Ciò consente un'allocazione più flessibile e riduce il consumo di memoria della tabella stessa.

Quando un sistema ha troppi page fault in rapida successione, trascorre più tempo a caricare e scaricare pagine che a eseguire processi. Questo fenomeno, detto **thrashing**, degrada fortemente le prestazioni. È tipico dei sistemi sovraccarichi e può essere mitigato da tecniche di gestione avanzata della memoria come il working set.

1.10 Descrivere la segmentazione

La **segmentazione** è una tecnica di gestione della memoria che suddivide lo spazio di indirizzamento di un processo in più *segmenti logici*, ciascuno corrispondente a un'unità semantica distinta (come codice, dati, stack, heap, ecc.). Ogni segmento può avere dimensioni diverse, essere allocato in modo indipendente e avere specifici diritti di accesso.

Un indirizzo logico in un sistema segmentato ha la forma $\langle s, d \rangle$, dove:

- s è l'indice del segmento,
- d è lo spostamento (offset) all'interno del segmento.

Per ogni processo, il sistema mantiene una propria **tabella dei segmenti**, in cui ogni entry contiene:

- la **base**, ovvero l'indirizzo fisico di inizio del segmento;

- il **limite**, ovvero la dimensione massima del segmento.

L'indirizzo della tabella dei segmenti per il processo corrente è conservato nel registro speciale **STBR** (Segment Table Base Register).

Durante l'accesso alla memoria, il sistema operativo verifica che $d < \text{limite}$: se la condizione è soddisfatta, l'indirizzo fisico viene calcolato sommando **base** e d .

La segmentazione semplice, pur separando logicamente le diverse aree di memoria, soffre degli stessi problemi dell'allocazione a dimensione variabile, come la *frammentazione esterna*. Per mitigare questo problema, si introduce la **segmentazione paginata**, che consiste nell'applicare la paginazione all'interno di ciascun segmento. In questo modo:

- ogni segmento è suddiviso in pagine di dimensione fissa,
- si riduce la frammentazione esterna,
- si conserva la separazione logica dei segmenti.

1.11 Descrivere la memoria virtuale

La **memoria virtuale** è una tecnica di gestione della memoria che consente ai processi di utilizzare uno spazio di indirizzamento apparentemente più grande della memoria fisica disponibile. Grazie a essa, un processo può essere eseguito anche se non completamente residente in memoria centrale, permettendo una maggiore efficienza nell'utilizzo delle risorse e un elevato grado di multiprogrammazione.

Questa tecnica viene implementata attraverso la **paginazione su richiesta**, che consiste nel caricare in memoria centrale soltanto le pagine effettivamente necessarie durante l'esecuzione del processo. Le restanti pagine rimangono temporaneamente memorizzate su disco e vengono caricate quando richieste.

La paginazione su richiesta prevede due fasi principali:

- **Pre-paging**: il *pager* può anticipare l'uso di determinate pagine caricandole preventivamente sulla base di previsioni.
- **Gestione a tempo di esecuzione**: durante l'esecuzione del processo, quando viene richiesto l'accesso a una pagina non presente in memoria fisica, si verifica un **page fault**. In tal caso, il sistema operativo interviene per caricare la pagina mancante.

Il **page fault** rappresenta un'interruzione causata dall'accesso a una pagina non presente in memoria. Alla sua occorrenza, il sistema operativo esegue le seguenti operazioni:

1. **Salvataggio** del contesto del processo interrotto;
2. **Verifica** della validità dell'indirizzo richiesto;
3. **Caricamento** della pagina dal disco in un frame libero;
4. **Aggiornamento** della tabella delle pagine del processo;
5. **Ripristino** del contesto e ripresa dell'esecuzione.

Nel caso in cui non vi siano frame liberi disponibili, si verifica una situazione di **sovrallocazione**. In questo caso, il sistema deve selezionare una **pagina vittima** da rimuovere dalla memoria per fare spazio a quella da caricare. Tale scelta viene effettuata tramite algoritmi di rimpiazzo delle pagine, come:

- LRU (Least Recently Used),
- LFU (Least Frequently Used),
- FIFO (First-In First-Out), ecc.

Per ottimizzare la scelta della pagina da espellere, ogni pagina può essere associata a un **dirty bit**, che indica se la pagina è stata modificata. Se il bit è attivo, la pagina dovrà essere salvata su disco prima di essere rimossa; in caso contrario, può essere semplicemente sovrascritta, riducendo il costo dell'operazione.

1.12 Descrivere le operazioni svolte dopo un Page Fault

Quando si verifica un **page fault**, il sistema operativo interviene per gestire la mancanza della pagina richiesta in memoria fisica. Le operazioni principali sono:

1. **Salvataggio del contesto:** il sistema salva lo stato del processo in esecuzione.
2. **Verifica della causa del fault:**
 - **Accesso valido:** la pagina richiesta fa parte dello spazio di indirizzamento del processo ma non è attualmente caricata in memoria (bit di validità = 0).
 - **Accesso non valido:** la pagina non appartiene al processo o viola i diritti di accesso. In tal caso il processo viene terminato con errore (es. **Segmentation Fault**).
3. **Gestione del caricamento della pagina:**
 - Se la memoria è piena, viene selezionata una **pagina vittima** (in base alla politica di rimpiazzo, es. LRU, FIFO).
 - Se la pagina vittima è stata modificata (bit **dirty** = 1), viene salvata in memoria secondaria.
 - La pagina richiesta viene quindi caricata dalla memoria secondaria (swap space) in un frame libero o rilasciato.
4. **Aggiornamento della tabella delle pagine:** la tabella del processo viene modificata per riflettere il nuovo stato della pagina (frame assegnato, bit di validità = 1, azzeramento del bit **dirty** se necessario).
5. **Ripristino del contesto ed esecuzione:** il sistema ripristina lo stato del processo e riavvia l'istruzione interrotta al momento del fault.

Questo meccanismo è gestito dal **pager**, un modulo del sistema operativo incaricato della politica di gestione della memoria virtuale. In condizioni di carico eccessivo, un numero elevato di page fault può generare fenomeni di **trashing**, dove il sistema trascorre più tempo a gestire le sostituzioni di pagina che ad eseguire effettivamente i processi.

1.13 Descrivere la sovrallocazione e gli algoritmi di sostituzione

In condizioni di **sovrallocazione**, ovvero quando la memoria fisica è interamente occupata e un processo richiede il caricamento di una nuova pagina, il sistema operativo deve scegliere una **pagina vittima** da rimuovere per fare spazio. Questa scelta è compito del **pager**, il componente responsabile della gestione della memoria virtuale.

Per ottimizzare il processo, ogni pagina caricata in memoria è associata a un **dirty bit**:

- **Dirty bit = 1:** la pagina è stata modificata e deve essere salvata su disco prima della rimozione.
- **Dirty bit = 0:** la pagina non è stata modificata e può essere scartata senza scrittura su disco.

Algoritmi di selezione della pagina vittima

Prima di esaminare il dirty bit, il sistema deve determinare un insieme di pagine **candidate alla sostituzione**. Gli algoritmi più comuni sono:

- **Least Frequently Used (LFU):** seleziona la pagina con il minor numero di accessi nel tempo. Richiede un contatore di accessi per ogni pagina.
- **Least Recently Used (LRU):** sceglie la pagina che non è stata utilizzata da più tempo. Si basa sul principio che le pagine utilizzate recentemente verranno usate anche nel prossimo futuro.
- **First-In-First-Out (FIFO):** elimina la pagina caricata da più tempo, indipendentemente dall'utilizzo recente. È semplice ma può essere inefficiente.

Implementazione di LRU

L'algoritmo LRU, pur essendo efficace, ha un costo computazionale elevato. Le principali implementazioni sono:

- **Time-stamping:** ogni pagina ha associato un timestamp aggiornato a ogni accesso; viene scelta la pagina col timestamp più vecchio.

- **Stack:** viene mantenuto uno stack delle pagine ordinate per accesso; a ogni accesso la pagina viene spostata in cima. La pagina in fondo è la meno recentemente usata.

Entrambe le tecniche richiedono un overhead significativo in termini di tempo e spazio.

1.14 Descrivere i metodi di accesso e di allocazione nel file system

Nel file system, i metodi di accesso definiscono come un processo può leggere o scrivere i dati di un file. Le principali tecniche sono:

- **Accesso sequenziale:** il file è visto come una sequenza lineare di record logici. L'accesso avviene in ordine, dalla prima all'ultima posizione. È il metodo più semplice e comune.
- **Accesso diretto:** il file è composto da record numerati e ogni record può essere letto o scritto direttamente, conoscendone l'indice. È ideale per basi di dati e file di grandi dimensioni a struttura fissa.
- **Accesso indicizzato:** viene utilizzata una struttura ad indice (simile a quella dei database) che mappa posizioni logiche a blocchi fisici. Permette un accesso efficiente anche su file di grandi dimensioni e supporta accesso sia sequenziale che diretto.

Tecniche di allocazione dei file

Le tecniche di allocazione gestiscono come i blocchi fisici su disco sono associati al contenuto di un file. Le principali sono:

- **Allocazione contigua:** tutti i blocchi del file sono memorizzati in posizioni fisicamente contigue sul disco.
 - *Vantaggi:* accesso molto veloce, semplice da implementare.
 - *Svantaggi:* frammentazione esterna, difficoltà nell'estensione del file.
- **Allocazione a lista concatenata:** ogni blocco contiene un puntatore al blocco successivo.
 - *Vantaggi:* nessuna frammentazione esterna, facile espansione.
 - *Svantaggi:* accesso lento (soprattutto diretto), maggiore spazio usato per i puntatori.
- **Allocazione tramite File Allocation Table (FAT):** tutti i puntatori ai blocchi sono memorizzati in una tabella centrale in RAM. Ogni voce della tabella indica il blocco successivo del file.
 - *Vantaggi:* migliora l'accesso rispetto alla lista concatenata, particolarmente efficiente per piccoli file system.
 - *Svantaggi:* la tabella può diventare molto grande, soprattutto con dischi ampi.
- **Allocazione indicizzata:** per ogni file esiste un blocco indice che contiene tutti i puntatori ai blocchi dati del file.
 - *Vantaggi:* supporta accesso diretto, gestisce bene file grandi.
 - *Svantaggi:* gestione complessa se il file è molto grande (si usano indici multilivello, come in UNIX con gli i-node).

1.15 Descrivere l'organizzazione fisica del file system in Unix

Nel file system Unix, tutto è rappresentato come file, che si suddividono in tre categorie principali:

- **File ordinari:** contengono dati o programmi.
- **Direttori:** contengono riferimenti ad altri file o directory (nomi e i-number).
- **File speciali:** rappresentano dispositivi fisici (carattere o blocco).

Ogni file è identificato univocamente da un **i-node** (index node), strutture dati che descrivono i file a livello fisico. Più nomi (link) possono riferirsi allo stesso i-node attraverso il file system.

Il file system è organizzato su disco in blocchi strutturati:

1. **Boot block:** contiene il codice di avvio del sistema (boot loader).

2. **Superblock:** memorizza informazioni globali sul file system, come dimensioni, numero di i-node, lista dei blocchi liberi, ecc.
3. **i-list:** contiene tutti gli i-node, ognuno identificato da un **i-number**.
4. **Blocchi dati:** contengono il contenuto reale dei file.

Ogni **i-node** è un descrittore del file e include:

- Tipo di file (ordinario, directory, speciale)
- Proprietario (UID) e gruppo (GID)
- Permessi di accesso (12 bit)
- Dimensione del file in byte
- Data e ora (creazione, modifica, accesso)
- Numero di link al file
- Puntatori ai blocchi che contengono i dati

Poiché l'allocazione dei file non è contigua, gli i-node contengono una serie di puntatori ai blocchi del file:

- **Blocchi 1–10:** indirizzamento diretto (puntano direttamente ai blocchi dati)
- **Blocco 11:** indirizzamento indiretto (punta a un blocco contenente ulteriori indirizzi)
- **Blocco 12:** indirizzamento doppio indiretto (punta a un blocco che punta ad altri blocchi contenenti indirizzi)
- **Blocco 13:** indirizzamento triplo indiretto (punta a un blocco che punta ad altri blocchi, ecc.)

Questa struttura consente una gestione efficiente dei file piccoli tramite puntatori diretti, e supporta anche file di dimensioni elevate tramite livelli di indirizzamento.

1.16 Descrivi lo scheduling della CPU

Lo scheduling della CPU è il meccanismo mediante il quale il sistema operativo decide quale processo, tra quelli pronti all'esecuzione, debba ottenere l'accesso alla CPU. I processi possono essere classificati in due categorie:

- **CPU-bound:** fanno un uso intensivo della CPU.
- **I/O-bound:** passano gran parte del tempo in attesa di operazioni di input/output.

Gli algoritmi di scheduling si dividono in:

- **Preemptive:** il sistema operativo può interrompere un processo in esecuzione per assegnare la CPU a un altro (tipico nei sistemi time-sharing).
- **Non-preemptive:** una volta assegnata la CPU, il processo la mantiene fino al termine o alla richiesta di I/O.

Le decisioni di scheduling sono prese dallo **scheduler**, il cui obiettivo è:

- Massimizzare:
 - **Utilizzo della CPU**
 - **Throughput:** numero di processi completati per unità di tempo
- Minimizzare:
 - **Tempo di attesa:** tempo trascorso nella ready queue
 - **Turnaround time:** tempo totale tra la sottomissione e il completamento di un processo
 - **Tempo di risposta:** tempo tra la sottomissione e la prima risposta prodotta

I principali algoritmi di scheduling includono:

- **First-Come-First-Served (FCFS):** i processi vengono eseguiti nell'ordine di arrivo. Processi CPU-bound possono causare lunghe attese, mentre una sequenza di processi I/O-bound può portare a un basso utilizzo della CPU.

- **Shortest Job First (SJF)**: esegue per primo il processo con il tempo di esecuzione (CPU burst) più breve. Richiede una stima accurata dei burst, non sempre semplice da ottenere.

Introdurre il concetto di **priorità** permette di differenziare i processi in base all'importanza o all'urgenza, definendo così algoritmi più flessibili. Tra questi:

- **Round Robin (RR)**: assegna la CPU ai processi in modo ciclico per intervalli di tempo detti *quantum*. Se il quantum è troppo breve, si introduce overhead da frequenti context switch; se troppo lungo, peggiora il tempo di risposta.
- **Multilevel Feedback Queue (MLFQ)**: utilizza più code con priorità decrescente e i processi possono migrare tra le code. Le priorità vengono ricalcolate periodicamente, applicando un meccanismo di *feedback negativo* per penalizzare i processi CPU-bound.

Una volta selezionato il processo da eseguire, il **dispatcher** si occupa del *context switch*, ovvero del salvataggio dello stato corrente e del caricamento dello stato del nuovo processo da eseguire.

1.17 Descrivere i passaggi di sostituzione ed elaborazione nella Shell Bash

La shell Bash, prima di eseguire un comando, effettua una serie di passaggi di **sostituzione** ed **elaborazione** sull'input fornito. L'ordine di esecuzione di questi passaggi è fondamentale e avviene secondo le seguenti fasi:

1. **Espansione dei comandi**: le espressioni racchiuse tra apici inversi 'comando' vengono sostituite con l'output prodotto dall'esecuzione del comando specificato.
2. **Espansione delle variabili**: le variabili precedute dal simbolo \$ vengono sostituite con il loro valore corrente.
3. **Espansione dei metacaratteri**: i metacaratteri come *, ?, [] vengono sostituiti con i nomi dei file che corrispondono al pattern specificato.

1.18 Descrivere i Monitor e variabili condizione

Il **monitor** è un costrutto di alto livello utilizzato per la **sincronizzazione** nei sistemi concorrenti. Rappresenta un meccanismo di controllo dell'accesso a risorse condivise, garantendo che solo un processo alla volta possa eseguire codice critico all'interno del monitor stesso.

L'accesso alla risorsa condivisa avviene esclusivamente attraverso procedure speciali dette **entry**, e tale accesso è regolato da due vincoli fondamentali:

- Le entry devono essere **mutuamente esclusive**, ovvero un solo processo alla volta può eseguirle.
- Le entry devono essere **le uniche operazioni consentite** per accedere alla risorsa protetta.

La sincronizzazione nei monitor si articola su due livelli:

1. **Mutua esclusione**: viene garantita automaticamente dal monitor. Se un processo è già all'interno del monitor, gli altri vengono sospesi in una **entry queue** finché non possono accedere.
2. **Sincronizzazione condizionale**: viene gestita tramite **variabili condizione** che consentono ai processi di attendere che una certa condizione logica sia vera prima di procedere. Se la condizione non è soddisfatta, il processo viene sospeso in una **condition queue**.

Le **variabili condizione** sono meccanismi che permettono il controllo fine del flusso di esecuzione tra processi all'interno del monitor. Si usano due primitive fondamentali:

- **wait()**: il processo corrente viene sospeso e inserito nella condition queue associata.
- **signal()**: risveglia un processo sospeso sulla variabile condizione, se presente.

L'operazione **signal()** può essere gestita in due modalità principali:

- **signal_and_wait**: il processo che invoca la **signal** si sospende, cedendo immediatamente il controllo al processo risvegliato.
- **signal_and_continue**: il processo che invoca la **signal** continua la sua esecuzione; il processo risvegliato rientrerà nel monitor appena sarà possibile.

La presenza di una delle due modalità è necessaria a evitare la violazione della mutua esclusione nel risveglio di un processo.

1.19 Descrivi le operazioni e semantiche delle variabili condizione

Le **variabili condizione** sono utilizzate nei monitor per sincronizzare i processi in base a condizioni logiche definite nel programma. Esse consentono ad un processo di sospendersi finché una determinata condizione non diventa vera.

Le principali operazioni disponibili sono:

- **wait()**: sospende il processo corrente e lo inserisce nella **condition queue** associata alla variabile condizione. Il processo rilascia automaticamente il monitor, permettendo ad altri processi di accedervi.
- **signal()**: risveglia uno dei processi sospesi nella condition queue (se presente). Il comportamento successivo dipende dalla semantica scelta.

Esistono due semantiche principali per la gestione del **signal**:

- **signal_and_continue**:
 - Il processo che invoca **signal** **continua** la propria esecuzione all'interno del monitor.
 - Il processo risvegliato viene spostato nella **entry queue**, da cui potrà rientrare nel monitor quando sarà il suo turno.
 - Poiché nel frattempo la condizione potrebbe essere cambiata da altri processi, il processo risvegliato deve **ricontrollare la condizione** al momento della ripresa.
- **signal_and_wait**:
 - Il processo che invoca **signal** **cede immediatamente** il controllo del monitor al processo risvegliato.
 - Il risvegliato può riprendere l'esecuzione con la **garanzia che la condizione è ancora valida**, dato che nessun altro processo ha avuto accesso al monitor nel frattempo.
 - Il processo risvegliante viene inserito nella entry queue e attenderà il proprio turno.

1.20 Esporre possibili soluzioni al problema della mutua esclusione

La **mutua esclusione** è una proprietà volta a garantire che solo un processo per volta possa accedere a una **sezione critica**, ovvero una porzione di codice in cui si manipolano risorse condivise.

Affinché una soluzione alla mutua esclusione sia considerata corretta, essa deve garantire:

- **Mutua esclusione**: mai più di un processo alla volta può trovarsi nella sezione critica.
- **Accesso**: se nessun processo non si trova nella sezione critica, allora non può bloccarne l'accesso ad altri processi.
- **Assenza di deadlock**: non si devono verificare situazioni di stallo in cui i processi si attendono a vicenda.

Le possibili soluzioni si possono suddividere in tre categorie principali:

- **Soluzioni algoritmiche (software)**: si basano esclusivamente sulla cooperazione dei processi e sull'uso di variabili condivise, senza il supporto dell'hardware o del sistema operativo. Esempi noti:
 - **Algoritmo di Dekker**: primo algoritmo corretto per due processi, complesso e difficile da estendere.
 - **Algoritmo di Peterson**: più semplice e leggibile, anch'esso limitato a due processi ma spesso utilizzato a scopo didattico.

- **Soluzioni hardware:** sfruttano primitive fornite direttamente dall'architettura, come:
 - **Disabilitazione degli interrupt:** utilizzabile solo in sistemi monoprocesso.
 - **Istruzioni atomiche speciali,** come **test-and-set**, **compare-and-swap** o **exchange**, che permettono di eseguire controlli e modifiche su variabili in maniera indivisibile.
- **Soluzioni basate su strumenti di sincronizzazione (kernel-level):** fanno uso di primitive offerte dal sistema operativo per astrarre e semplificare la gestione della mutua esclusione. Strumenti principali:
 - **Semafori:** variabili speciali che permettono il controllo dell'accesso concorrente attraverso operazioni **wait** (P) e **signal** (V).
 - **Monitors:** costrutti ad alto livello che combinano mutua esclusione e variabili condizione.
 - **Mutex e lock:** meccanismi semplificati per garantire l'accesso esclusivo a sezioni di codice.

1.21 Definizione e uso di semaforo

Il **semaforo** è uno strumento di sincronizzazione fornito dal kernel per gestire la mutua esclusione e la sincronizzazione tra processi o thread in ambienti concorrenti, specialmente nel modello ad ambiente globale. Un semaforo è generalmente rappresentato da una variabile intera e da una **coda dei processi sospesi**, e può essere manipolato solo attraverso due primitive atomiche:

- **P(s)** (proberen): se il valore del semaforo è zero, il processo che ha eseguito l'operazione viene sospeso e inserito nella coda associata al semaforo, altrimenti può accedere.
- **V(s)** (verhogen): incrementa il valore del semaforo. Se il valore risultante è maggiore di zero, un processo dalla coda viene risvegliato.

Per essere correttamente utilizzato, un semaforo deve soddisfare alcuni requisiti fondamentali:

- **Assenza di attesa attiva** (*busy waiting*): i processi che non possono procedere devono essere messi in attesa passiva nella coda del semaforo, liberando la CPU.
- **Assenza di starvation:** l'ordine di risveglio dei processi sospesi deve essere equo, tipicamente attraverso una politica *FIFO*, per evitare che alcuni processi vengano ignorati indefinitamente.
- **Atomicità delle operazioni:** le primitive P e V devono essere indivisibili, cioè non possono essere interrotte da altri processi o thread.

I semafori sono strumenti in grado di risolvere sia problemi di **mutua esclusione** che di **sincronizzazione**.

1.22 Descrizione e confronto tra gestione a controllo di programma e gestione a interruzioni di dispositivi di I/O

La gestione dei dispositivi di I/O avviene tramite interfacce standardizzate che nascondono la complessità specifica di ciascun dispositivo, consentendo un accesso uniforme e corretto. L'accesso alle periferiche è mediato da un **controllore**, che si interpone tra CPU e dispositivo, gestendo il trasferimento dei dati.

Il controllore è dotato di una serie di registri, accessibili dalla CPU, tramite i quali è possibile interagire con il dispositivo:

- **Registro di controllo:** memorizza i comandi da inviare al dispositivo, compreso il bit di start per attivarne il funzionamento.
- **Registro di stato:** indica lo stato corrente del dispositivo, inclusa l'informazione di completamento delle operazioni.
- **Registro dei dati:** funge da buffer per i dati in ingresso o in uscita.

Si distinguono due categorie di processi:

- **Processi interni:** eseguiti dalla CPU.
- **Processi esterni:** eseguiti dal controllore.

Per permettere la cooperazione tra processi interni ed esterni si adottano due tecniche principali:

- **Gestione a controllo di programma (polling):**
 - Il processo interno (CPU) esegue ciclicamente controlli sul registro di stato del dispositivo per verificarne la disponibilità.
 - Quando lo stato indica che il dispositivo è pronto, la CPU procede con l'operazione di I/O.
 - Questo approccio introduce **attesa attiva** (busy waiting), sprecando tempo di CPU e risultando inefficiente in ambienti multiprogrammati.
- **Gestione a interruzioni:**
 - La CPU invia il comando di I/O al controllore e continua l'esecuzione del programma.
 - Quando il dispositivo ha completato l'operazione, genera un'interruzione (interrupt).
 - L'interrupt handler viene invocato automaticamente, legge i dati o segnala il completamento, sbloccando eventualmente il processo in attesa.
 - In ambienti concorrenti, si utilizza un semaforo per ciascun dispositivo, sincronizzando i processi con le operazioni P (wait) e V (signal).

Confronto tra i due approcci

- **Controllo di programma:** semplice da implementare ma inefficiente, comporta spreco di cicli CPU dovuto all'attesa attiva. Adatto solo a sistemi semplici o dedicati.
- **Interruzioni:** più complesso da gestire ma consente di sfruttare al meglio la CPU. È la soluzione preferibile in sistemi multitasking o time-sharing.